# SOFTWARE DESIGN DOCUMENT

## for

# ZipClip

**Version 1.0**

**Prepared by : 1. ABHIJITH R (VJC22CSD003)**
**2. ANIRUDH KB (VJC22CSD012)**
**3. JESVIN J (VJC22CSD032)**

**Submitted to :Ms. Sabitha Raju**
**HOD,CG.DEPT**

**October 9, 2025**

# Contents

# 1 Overview

## 1.1 Scope

The ”ZipClip” project is an AI-driven content creation tool designed to automatically convert long-form video content into short, engaging clips and to generate story reels from user photos. The scope includes a robust backend for multi-modal video analysis (visual, audio, and text), an interactive front-end dashboard for preview and editing, and export pipelines that produce platform-optimized outputs for YouTube Shorts, Instagram Reels, and TikTok. Stakeholder concerns—such as accuracy of highlight selection, processing latency, and export quality—are addressed through specific design views and documented elements, visualized using UML diagrams and adhering to IEEE 1016-2009 standards.

## 1.2 Purpose

This SDD provides a comprehensive blueprint for ZipClip’s design, describing the system architecture, component interactions, and development approach. It guides implementation, supports creation of test cases, facilitates future maintenance, and ensures the design fulfills functional and non-functional requirements from the SRS. The goal is to deliver a reliable, scalable system that automates highlight extraction and story reel creation while offering an intuitive editing experience.

## 1.3 Intended audience

This document targets the development team (Abhijith R, Anirudh K B, Sreejith V M), responsible for implementation and testing; the project guide (Mrs Susmitha John), overseeing progress; faculty at Viswajyothi College of Engineering and Technology (VJCET), evaluating the academic quality; and other stakeholders assessing the project’s technical feasibility and deployment considerations.

## 1.4 References

[1] IEEE. IEEE Std 1016-2009 IEEE Standard for Information Technology - System Design - Software Design Descriptions. IEEE Computer Society, 2009.

[2] Software Requirements Specification (SRS) Document of ZipClip prepared by Abhijith R, Anirudh K B, Jesvin J.

[3]YouTube Shorts documentation and developer resources: `https://developers.google.com/youtube/shorts`

# 2 Definitions, Acronyms and Abbreviations

This section defines key terms, acronyms, and abbreviations used in this document:

| Term | Definition |
|---|---|
| Block Diagram | Diagram showing system components and connections. |
| Class Diagram | UML diagram of classes, attributes, and relationships. |
| Clip Export | Process of rendering and packaging short-form video output. |
| FPS | Frames Per Second, target depends on preview playback (30-60 FPS). |
| IDE | Tools like VS Code or PyCharm for backend and AI development. |
| IEEE Standard | IEEE 1016-2009 for software design documentation. |
| CDN | Content Delivery Network for serving media assets efficiently. |
| SDD | Document detailing ZipClip's software design. |
| SDK | Third-party SDKs for video playback, cloud storage, or AI inference. |
| SRS | Document specifying ZipClip requirements. |
| Stakeholders | Non-developer interested parties (e.g., guide, evaluators). |
| FFmpeg | Command-line tool for video processing and encoding. |
| ASR | Automatic Speech Recognition for transcript generation. |
| UI | User interface for web/mobile dashboard. |
| Use Case Diagram | UML diagram of user-system interactions. |
| User | Creator or consumer using ZipClip to generate clips or reels. |
| ML | Machine Learning models used for content scoring and detection. |
| NLP | Natural Language Processing for transcript analysis and prompts. |

# 3 Conceptual Model for Software Design Descriptions

This section introduces ZipClip's foundational terms, design concepts, and SDD context, enhancing understanding of the system's terminology and its place within the software development lifecycle.

## 3.1 Software Design in Context

ZipClip is developed as a full-stack application combining a cross-platform front end (web and/or mobile) with a backend orchestrating AI pipelines. The AI layer performs video frame analysis, audio transcription, emotion detection, and multimodal fusion to score and rank highlights. Development uses Python for AI modules, Node.js or Django for backend services, and Flutter or React for front-end interfaces. The design prioritizes modularity for independent development of AI agents, scalability for processing large media files, and responsiveness for user previews.

## 3.2 Software Design Descriptions within the Life Cycle

Following IEEE standards, ZipClip's lifecycle includes requirements analysis (via SRS), design description (this SDD), implementation of AI pipelines and UI components, and verification/validation to ensure accuracy of highlight detection and export quality.

### 3.2.1 Influences on SDD Preparation

The SRS for ZipClip drives this design document by specifying functional requirements (e.g., upload, highlight generation, photo-to-reel conversion), non-functional goals (e.g., latency targets, robustness), and user interface expectations. Stakeholder feedback—especially from creators and educators—will refine UI/UX and prioritization of features during development.

### 3.2.2 Influences on Software Life Cycle Products

Early prototypes and demo clips will be shared with stakeholders for iterative feedback. This SDD aims to keep the design traceable and adaptable, supporting deployment to cloud platforms for scalable processing and enabling future integration with content platforms and social APIs.

### 3.2.3 Design Verification and Design Role in Validation

Verification confirms that ZipClip's implementation matches SRS-specified behaviors (e.g., correct detection of scene boundaries, accurate transcript generation), using unit and integration tests. Validation ensures the end product meets user needs—generating engaging, contextually consistent clips—through user testing and stakeholder review.

# 4 Design Description Information Content

## 4.1 Introduction

This section provides a detailed exploration of ZipClip's design and implementation strategy. ZipClip automates highlight extraction from long videos and produces story reels from user photos by integrating computer vision, audio analysis, and NLP. The design covers identification of subsystems, stakeholder concerns, design views and viewpoints, elements, design rationale, and chosen implementation languages and tools. The goal is to provide developers a clear roadmap to implement a reliable, extensible system that balances accuracy, performance, and usability.

## 4.2 SDD Identification

ZipClip is assigned a unique identifier, "ZC-SDD-1.0," to track versions, updates, and changes throughout its development lifecycle. The project timeline targets a deliverable demo by March 31, 2026, with intermediate prototypes showcasing core AI-driven highlight extraction and photo-to-reel generation. The document references the SRS and other materials for requirements and test planning.

## 4.3 Design Stakeholders and Their Concerns

Key stakeholders include the development team (Abhijith R, Anirudh K B, Jesvin J) and guide (Mrs Sabitha Raju) at VJCET's Department of Computer Science and Design. Stakeholder concerns include accuracy of highlight detection, low processing latency for previews, clarity of the interactive dashboard, export quality for social platforms, and maintainability of AI models and backend services.

## 4.4 Design Views

Design views focus attention on system aspects critical to ZipClip's success by providing distinct perspectives on its architecture. The context view, for example, examines the system's external interactions, detailing how users engage via the UI, how media is stored in cloud services like S3, and how final clips are exported to social platforms such as YouTube or TikTok. The composition view then breaks the internal system into its primary functional subsystems, including the ingestion service for handling uploads, the core AI analysis engine for processing, the clip assembly module for creating the final product, and the export service for transcoding. Each of these views is formally mapped to specific UML diagrams—such as block, component, and deployment diagrams—which provide a clear and unambiguous blueprint of the system's boundaries, component responsibilities, and critical integration points.

## 4.5 Design Viewpoints

TThis SDD describes a range of architectural viewpoints to provide a comprehensive system overview. The context viewpoint defines ZipClip's place in its operational environment, including interactions with external services like cloud storage, while the composition and logical viewpoints break the system down into its core components—such as the API gateway, AI workers, and database. The dependency viewpoint maps the relationships between these modules, and the interface viewpoint specifies the REST and gRPC APIs governing their communication. Furthermore, the structure viewpoint details the static organization of the codebase, which is complemented by the interaction viewpoint that models the dynamic runtime behavior. Each of these perspectives helps clarify critical concerns, from ensuring efficient data flow for media and orchestrating AI inference jobs, to achieving the UI responsiveness required for an intuitive content creation experience.

## 4.6 Design Elements

Design elements include entities such as User, MediaAsset, AIJob, Clip, and Project. Relationships specify flows (e.g., MediaAsset uploaded triggers an AIJob). Attributes capture metadata (e.g., duration, format, language) and constraints (e.g., file size limits, processing timeout). These elements form the backbone of ZipClip's architecture and guide implementation details.

## 4.7 Design Overlays

Design overlays provide additional runtime details, such as processing metrics like queue length and job latency for performance monitoring. The system also incorporates robust logging strategies, including structured per-job logs and centralized error reporting. To enhance performance, various optimization strategies are employed, such as caching pretrained models to reduce latency and using chunked processing for large files.

## 4.8 Design Rationale

The design rationale centers on a modular architecture chosen to isolate responsibilities for tasks like scene detection and audio transcription, which improves fault tolerance and allows for independent development. This is supported by using Python for AI modules to access robust ML tooling, while a Node.js or Django backend handles API orchestration. Frontend choices like Flutter or React are prioritized to ensure a cross-platform, responsive dashboard.

## 4.9 Design Languages

UThe project uses UML to model the system's static and dynamic aspects. Implementation languages are chosen for their specific strengths, with Python used for AI processing, JavaScript/TypeScript for backend services, and Dart/JavaScript for frontend clients. System integration relies on REST or gRPC APIs for communication, while asynchronous job queues like Celery/RabbitMQ manage long-running tasks to keep the application responsive.

# 5 Design Viewpoints

## 5.1 Introduction

This section presents ZipClip's key design viewpoints, detailing concerns, elements, and UML examples to guide development. Each viewpoint ensures that AI pipelines, user interfaces, and backend services integrate seamlessly to provide an efficient workflow for content creators and casual users alike.

## 5.2 Context Viewpoint

The context viewpoint illustrates ZipClip's interactions with users and external services, emphasizing media ingestion, AI processing, and content export.

### 5.2.1 Design Concerns

Primary concerns include reliable upload and storage of large media files, secure handling of user data, timely processing for preview generation, and compatibility with third-party platforms for exporting final clips. Ensuring clear system boundaries and fallback mechanisms (e.g., retry on failed uploads) is critical.

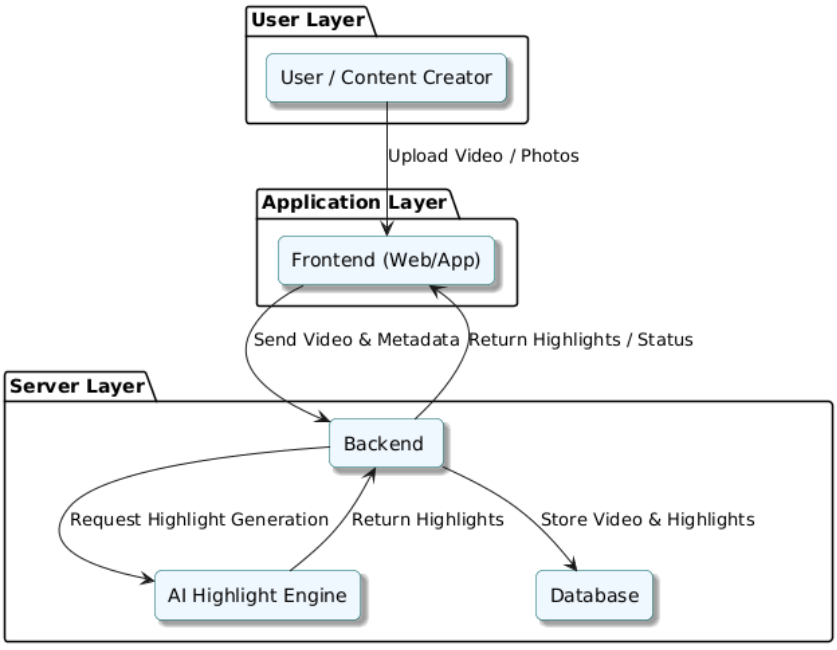


Figure 5.1: Block Diagram of ZipClip

### 5.2.2 Design Elements

Entities include Users (uploading videos/photos), Cloud Storage (storing media), AI Services (scene/audio/emotion analysis), and Export Services (formatting and publishing). Relationships detail triggers and data flows—for example, a successful upload triggers an AI Job that returns candidate clip timestamps to the dashboard.

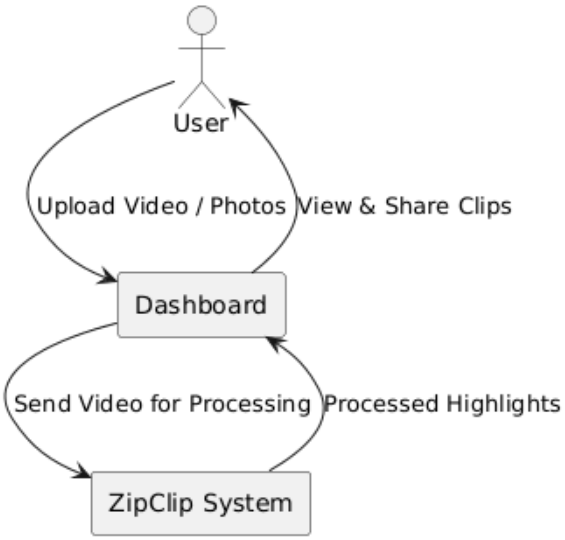


Figure 5.2: Context Diagram of ZipClip

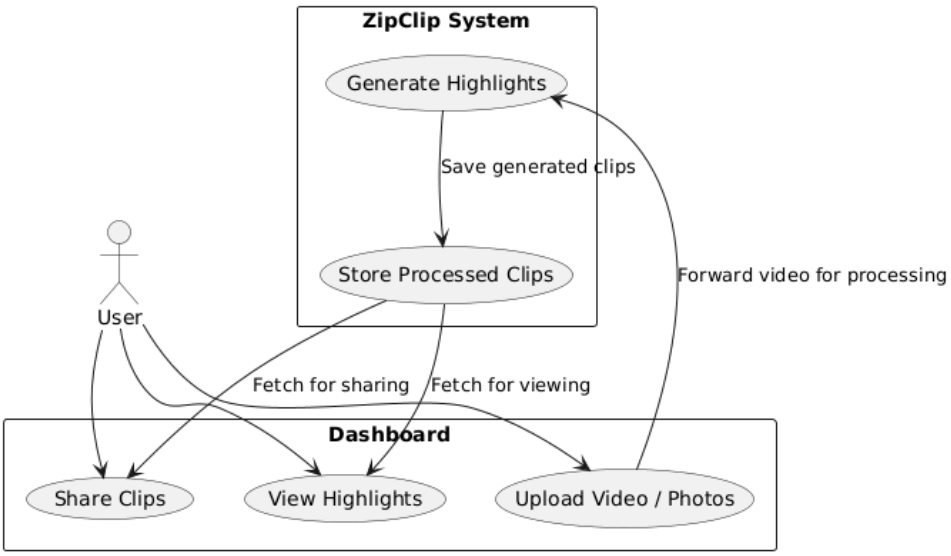


Figure 5.3: Use Case Diagram of ZipClip

### 5.2.3 Example Languages

UML block, context, and use case diagrams illustrate interactions among user-facing clients, storage components, and AI processing modules. Backend services handle orchestration and integration with third-party APIs for platform publishing.

## 5.3 Composition Viewpoint

Defines ZipClip's structure as a set of interconnected subsystems for scalable development and deployment.

### 5.3.1 Design Concerns

Concerns include ensuring subsystem interoperability (ingestion, AI processing, clip assembly, export), horizontal scalability for AI inference, and fault containment to prevent single-component failures from halting the pipeline.

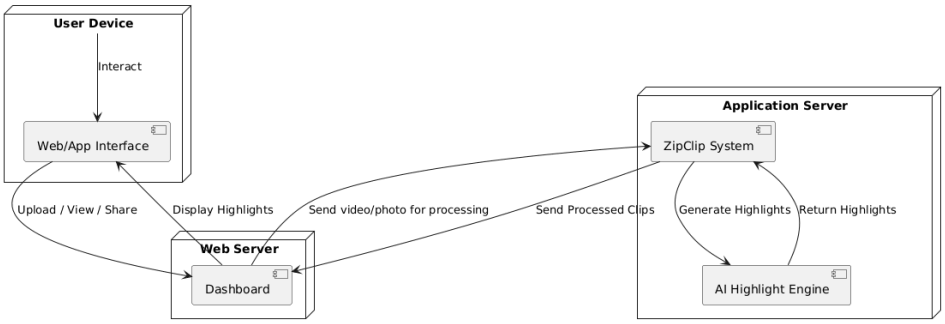


Figure 5.4: Deployment Diagram of ZipClip

### 5.3.2 Design Elements

Submodules include Ingestion (upload and validation), AI Analysis (scene/audio/emotion agents), Clip Assembler (stitching and formatting), UI Dashboard, and Exporter. Attributes such as scaling policy and resource constraints are defined to maintain service availability and performance.

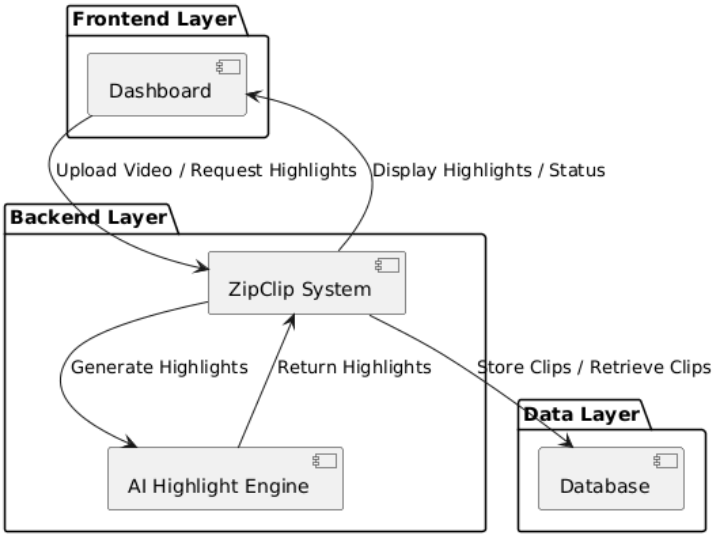


Figure 5.5: Component Diagram of ZipClip

### 5.3.3 Example Languages

UML deployment and component diagrams depict the logical decomposition. Implementation leverages containerization (Docker) and orchestration (Kubernetes) for production deployments.

## 5.4 Logical Viewpoint

Details ZipClip's class structures and static relationships, forming the core of the application logic.

### 5.4.1 Design Concerns

This viewpoint focuses on maintainable, testable code architecture. Concerns include encapsulation of AI models behind service interfaces, reuse of components for varying media types, and efficient metadata management for search and retrieval of generated clips.

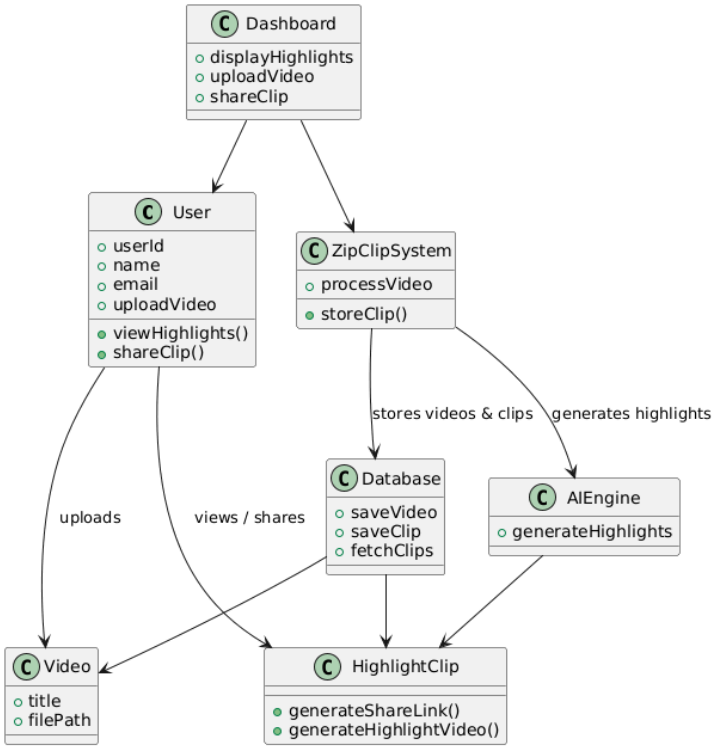


Figure 5.6: Class Diagram of ZipClip

### 5.4.2 Design Elements

Key classes include User, MediaAsset (attributes: duration, format), AIJob (attributes: type, status), Clip (attributes: start, end, score), and Project (grouping resources). Methods provide CRUD operations, job submission, and clip retrieval.

### 5.4.3 Example Languages

UML class diagrams illustrate these class structures. These classes are implemented across the backend (Node.js/Django) and service wrappers for AI modules (Python), exposing stable APIs for the frontend.

## 5.5 Dependency Viewpoint

Outlines dependencies among components to ensure reliable integration and operation.

### 5.5.1 Design Concerns

Key concerns include minimizing cascading failures, ensuring version compatibility for AI models and libraries, and establishing clear contracts between services (API schemas). Strategies for graceful degradation and retry mechanisms are essential.

### 5.5.2 Design Elements

Dependencies include UI depending on backend APIs for job status, AI modules depending on model artifacts and GPU resources, and exporter depending on cloud storage availability. Constraints and priority ordering prevent circular dependencies and manage resource contention.

### 5.5.3 Example Languages

UML component and package diagrams highlight these dependencies, and event-driven patterns (message queues) are recommended for decoupling components.

## 5.6 Interface Viewpoint

Specifies external and internal interfaces to ensure seamless interactions and usability.

### 5.6.1 Design Concerns

This viewpoint addresses usability of the dashboard, API contract stability, error handling for failed jobs, and accessibility features (language options, captions). Performance SLAs for preview generation and export must be defined and monitored.

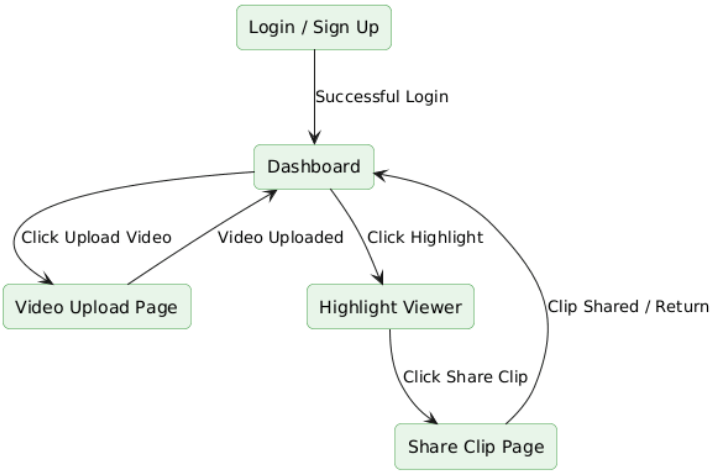


Figure 5.7: User Interface of ZipClip

### 5.6.2 Design Elements

Includes front-end UI endpoints (upload, preview, edit, export), backend REST/gRPC endpoints, and AI inference endpoints. Attributes: acceptable latency (¡5s for short preview thumbnails), reliability targets, and constraints such as maximum upload size and supported codecs.

### 5.6.3 Example Languages

UML use case and sequence diagrams are used to specify interface contracts and user flows. Frontend uses modern web/mobile frameworks to provide responsive editing and preview capabilities.

## 5.7 Structure Viewpoint

Describes package-level interactions to define architectural cohesion.

### 5.7.1 Design Concerns

This viewpoint focuses on modularity for independent updates, performance considerations for media processing, and measures to limit inter-package overhead to maintain responsiveness.

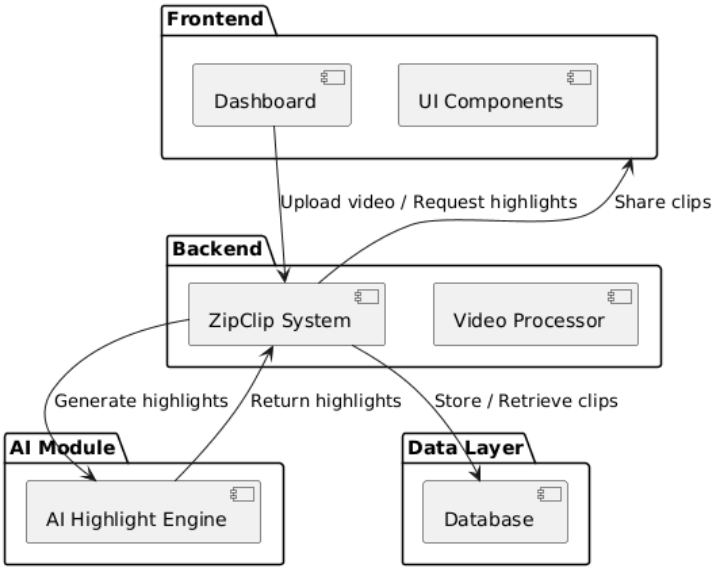


Figure 5.8: Package Diagram of ZipClip

### 5.7.2 Design Elements

Packages include Ingestion, AI Services, Clip Assembler, Exporter, and Dashboard. Attributes include package initialization time and inter-package call frequency. Relationships define data flow and service dependencies with constraints to preserve performance.

### 5.7.3 Example Languages

UML package diagrams provide a high-level view for developers and architects, and implementation uses package managers and namespaces for code organization.

## 5.8 Interaction Viewpoint

Details action sequences to model dynamic workflows in ZipClip.

### 5.8.1 Design Concerns

This viewpoint ensures responsive interactions and clear feedback loops, focusing on sequence timing for job submission, preview rendering, and export. It also addresses error recovery and user-driven adjustments to generated clips.

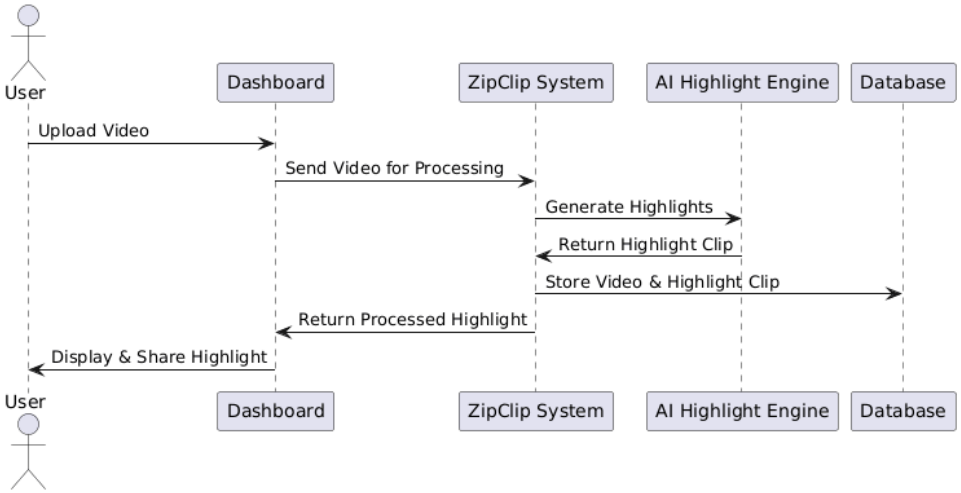


Figure 5.9: Sequence Diagram of ZipClip

### 5.8.2 Design Elements

Covers flows such as upload → AI analysis → preview → edit → export, with attributes like acceptable latencies and sequence durations. Constraints limit concurrent processing per user to avoid resource exhaustion.

### 5.8.3 Example Languages

UML sequence diagrams illustrate the temporal flow of interactions. Backend implementation uses asynchronous tasks and progress events to provide near-real-time status updates.

## 5.9 Static Dynamics Viewpoint

Models ZipClip's states and transitions to define dynamic operations.

### 5.9.1 Design Concerns

This viewpoint addresses state persistence for projects and clips, handling interruptions (e.g., failed jobs), and ensuring consistent transitions between states such as Idle, Processing, Preview Ready, and Exported.

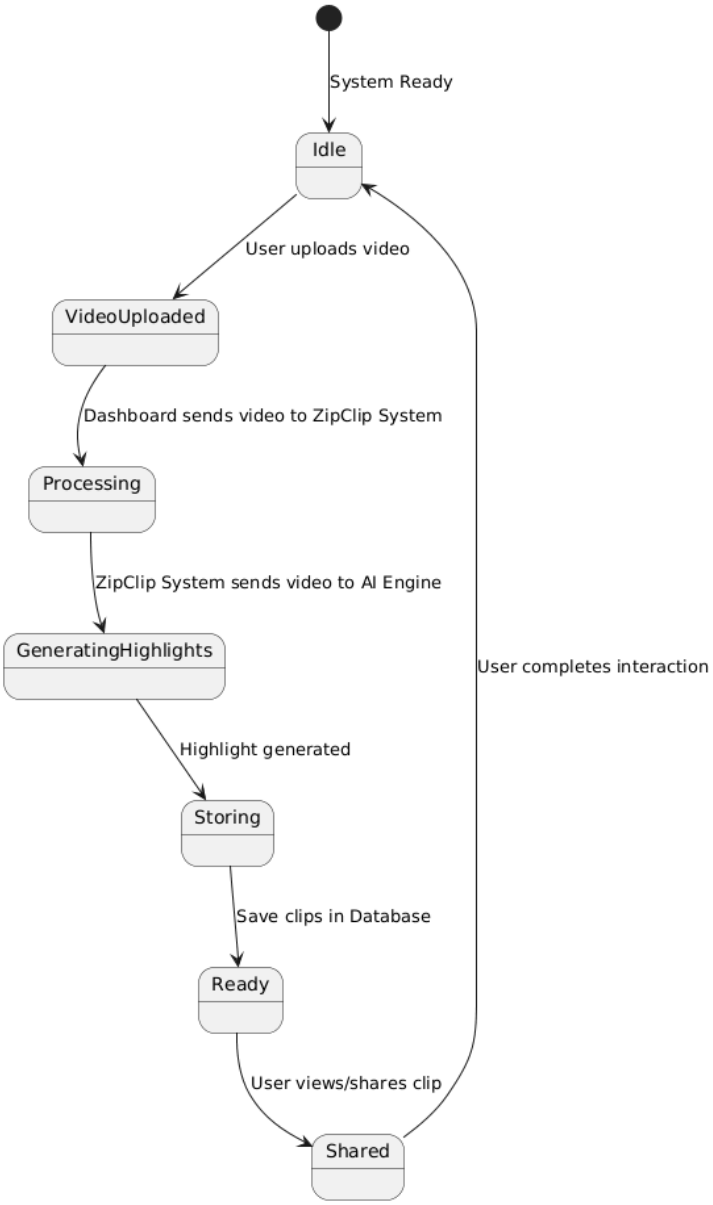


Figure 5.10: State Machine Diagram of ZipClip

### 5.9.2 Design Elements

States include Idle (no active jobs), Processing (AI job running), Preview Ready (generated candidates available), Editing (user modifying clips), and Exported (final clip produced). Transitions specify triggers and timing constraints to ensure a smooth user experience.

### 5.9.3 Example Languages

UML state diagrams provide clear visualization for developers implementing state management and persistence mechanisms. Asynchronous job handlers and state stores maintain consistency across restarts and failures.